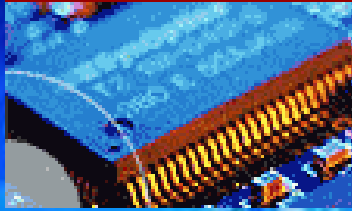


EEE3096S



**Embedded
Systems II**



**Prac2: Assembly
& Suggested Approach**

Lecture P3

Embedded Systems II

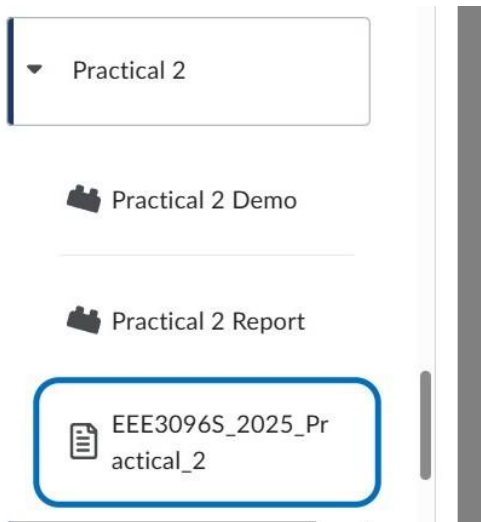
Dr Simon Winberg



Electrical Engineering
University of Cape Town

Prac2 : Important Reminders

- Prac2 released
- Comprises:
 - Demo part & Report Part
 - Separate assignments



EEE3095/6S: Practical 2

Embedded ARM Assembly Programming

August 13, 2025

1 Overview

An Assembly program is made up of a sequence of instructions, each of which gets executed by the CPU. But why bother learning Assembly when we have C for embedded programming?

The main reason nowadays is that Assembly code is usually used as a tiny piece of a (larger) C program to do something that is not easily done in C. And in this course, it is the task to

Prac2 : Reminders & Advice

- Doing everything in assembly
- However... you're recommended to follow this approach for both gaining sufficient understanding of assembly but also helping to get unstuck (if you're struggling to get the code assembly or working)

Prac2

- First off the “ARM-ASM-Tutorial” can certainly be of much help in learning assembly thoroughly
- The Open On-Chip Debugger (OpenOCD) can be worth trying (see <https://openocd.org/>)
but for Prac2 you are meant to
develop code for STM32CubeIDE
(Can't really port projects easily between these)

Prac2 – suggested starting approach

- For the initial tasks (1 – 3) you should really try to do your assembly from scratch, as in
 - Plan using e.g. pseudo code or just a rough start to the ARM instructions
 - Go through the process of selecting the instructions to use (e.g. `ldrb r1,[r0,#0]` etc).
 - You could do paper-based ‘dry run’ or ‘instruction tracing’ (although this is more if you’re not at a computer or in a testing situation)

Dry-run of assembly program

- This is where you simply have a table that indicates the instruction and the registers.
- The instructions are set in the sequence that they will run
- You go through the process of showing how the various registers (and memory addresses) change line-by-line as the program runs
- See next slide ...

Dry Run Example : plan some code to try

- Here's pseudocode for assembly we're going to dry run

Function: `IS_INPUT_ZERO`

Purpose: Checks a memory-mapped input port.

Returns 1 if input byte is zero, or 0 otherwise.

Return register as per ARM convention using R0

Begin

1. Load address of input port `0xFFFF0004` into register R5.
2. Load 8-bit byte from address pointed to by R5 into R1.
3. Compare the value in R1 with 0.
This updates processor's status flags.
4. If result of the comparison was *equal* (i.e., zero flag is set), then move value 1 into return register R0.
5. If the comparison was *not equal* (i.e., zero flag is clear), move value 0 into the return register R0.
6. Return from the function.

End

NOTE!! `NOV` does not by default change the status flags. `MOVS` does change the status flags (so `MOV R0,1` doesn't change zero flag, but `MOVS R0,1` sets it to 0).

Dry Run Example : Let's write some asm

Converted the pseudocode to ARM assembly:

@ GNU ARM assembler (GAS)

.global IS_INPUT_ZERO @ make function name globally accessible

IS_INPUT_ZERO: @ label for starting address of this function

@ 1. Load address of the input port into R5

LDR R5, =0xFFFF0004

@ 2. Load a byte from address in R5 into R1

LDRB R1, [R5]

@ 3. Compare value in R1 with 0. This sets the Z flag if R1 is 0.

CMP R1, #0

@ 4. If Equal (Z flag set) then move 1 into R0

MOVEQ R0, #1 @ we use a conditional instruction to do this

@ MOV doesn't change status flags, but MOVS and MOVEQS does.

@ 5. If Not Equal (Z flag clear) then move 0 into R0

MOVNE R0, #0

@ 6. Return to the calling function

BX LR

Dry Run Example : the dry run table

Case where input read is 0 ...

PC	Instruction	Action & Explanation	R0	R1	R5	Z Flag
? initial state		IS_INPUT_ZERO is called.	?	?	?	0
0x1000	LDR R5, =0xFFFF0004	Loads 32-bit address of the port into R5.	?	?	0xFFFF0004	0
0x1004	LDRB R1, [R5]	Loads byte from memory at 0xFFFF0004 into R1. We assume this byte is 0x00.	?	0x00000000	0xFFFF0004	0
0x1008	CMP R1, #0	Compares R1 (which is 0) with 0. Since equal, the Z (Zero) flag gets set to 1.	?	0x00000000	0xFFFF0004	1
0x100C	MOVEQ R0, #1	This executes because EQ (Equal) condition is met (Z flag is 1). Moves 1 into R0.	0x00000001	0x00000000	0xFFFF0004	1
0x1010	MOVNE R0, #0	Skipped because the NE (Not Equal) condition is false (Z flag is 1). R0 remains unchanged.	0x00000001	0x00000000	0xFFFF0004	1
0x1014	BX LR	Branches to the return address stored in LR. The function is complete.	0x00000001	0x00000000	0xFFFF0004	1

Note: I like either using a different colour (e.g. red) to indicate the register(s) that changed when the instruction executed (or shading the cell for registers updated in that line).



... can be a rather lengthy and resource-consuming process

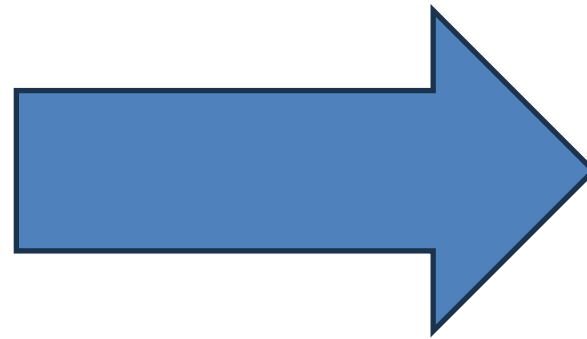
Prac2 – further along

- You **don't have to rely entirely on working from scratch** in the way just covered
- But you **should be able to do that**, as tests and exams will check this ability.
- Further along, you can rely (to some extent) on a compiler to assist ... to more help you check and improve your code.
- Don't just rely on using a compiler and having it generate assembly for you!!

Help from simulator and/or compiler

- OakSim can be of much use (can be quicker than compile and run gdb for a quick test)
- OakSim link:
<https://wunkolo.github.io/OakSim/>
- Can setting up compile flags to generate assembly that you can inspect
- Godbolt.org: allows for a C or C++ file to be converted to assembly and simulated, make sure to select suitable ARM GCC compiler*.
<https://godbolt.org/>

* Want a variant like arm-none-eabi-gcc version 10 or higher.



back to main part of L09